# Assemblies in .NET



Soyeb Ghachi

Technical Trainer | Mentor |SME

# Overview



What is Assembly?

Key Points about Assemblies .NET

Types of Assemblies in .NET

How to create and refer Private Assembly?

# What is Assembly?

- In .NET, an assembly is a unit of deployment and versioning for managed code. It is a compiled output of code that contains executable code, resources, metadata, and other information necessary for the execution of a program. Assemblies are the building blocks of .NET applications and libraries.
- Assemblies play a crucial role in the execution of .NET applications and libraries. They provide a way to package and distribute code, facilitate code reuse, versioning, and maintain metadata about types and dependencies.

# Key Points about Assemblies in .NET

- **File Types:** Assemblies are typically stored as files with the extension **.dll (dynamic-link library)** or **.exe (executable)**. A DLL is a library that contains reusable code, while an EXE is an executable application.
- **Units of Deployment:** Assemblies provide a way to package and distribute code. They can be deployed and referenced by other applications or assemblies, allowing code reuse and modularity.
- **Metadata:** Assemblies contain metadata that describes the **types**, **resources**, and **dependencies** within the assembly. This metadata is used by the .NET runtime for various purposes, such as type resolution, **versioning**, and **security**.

- **Versioning:** Assemblies have version numbers to enable side-by-side execution of different versions of the same assembly. Versioning allows applications to reference specific versions of assemblies and helps ensure backward compatibility.
- **Strong Naming:** Assemblies can be given a strong name, which consists of a cryptographic hash of the assembly's contents and a public/private key pair. Strong naming provides a unique identity for an assembly and ensures its integrity and authenticity.
- **Global Assembly Cache (GAC):** The GAC is a machine-wide repository for shared assemblies. It allows multiple applications to reference and share the same version of an assembly. Assemblies in the GAC are typically strongly named.

- **Assembly Manifest:** Each assembly contains an assembly manifest, which is a metadata section that describes the assembly's contents and dependencies. The manifest includes information such as the assembly's **name**, **version**, **culture**, referenced **assemblies**, and **security permissions**.

- **Reflection:** .NET provides reflection APIs that allow runtime inspection and manipulation of assemblies. Reflection enables tasks such as dynamically loading assemblies, examining types and members, and invoking methods at runtime.

# Types of Assemblies in .NET

- **Private Assemblies:**

Private assemblies are typically used by a single application and are stored in the application's directory or a subdirectory. Each application can have its own private assemblies, and they are not shared across applications. Private assemblies are the default type of assembly when you build an application.

- **Shared Assemblies:**

Shared assemblies, also known as "strongly named assemblies," are intended to be shared and used by multiple applications. Shared assemblies have a strong name, which consists of a unique identity generated using a cryptographic hash of the assembly's contents and a public/private key pair. These assemblies are typically stored in the Global Assembly Cache (GAC), a machine-wide repository for shared assemblies.

- **Satellite Assemblies:**

Satellite assemblies are a type of assembly used for localization and resource management. They contain localized resources such as strings, images, or other culture-specific data. Satellite assemblies allow you to create multilingual applications without modifying the main application assembly.

Satellite assemblies are typically stored in language-specific subdirectories and can be deployed alongside the main application assembly. The .NET runtime automatically selects the appropriate satellite assembly based on the current culture or language settings of the application.

# How to create and refer Private Assembly?

## Step 1-To create Private Assembly.

- **Create Project of type class library and write code and build the project.**

```
namespace GSTAPI
{
   public class GST
   {
      public double ProAmount { get; set; }

      public double Per { get; set; }

      public GST(double ProAmount,double Per)
      {
         this.ProAmount = ProAmount;
         this.Per=Per;
      }

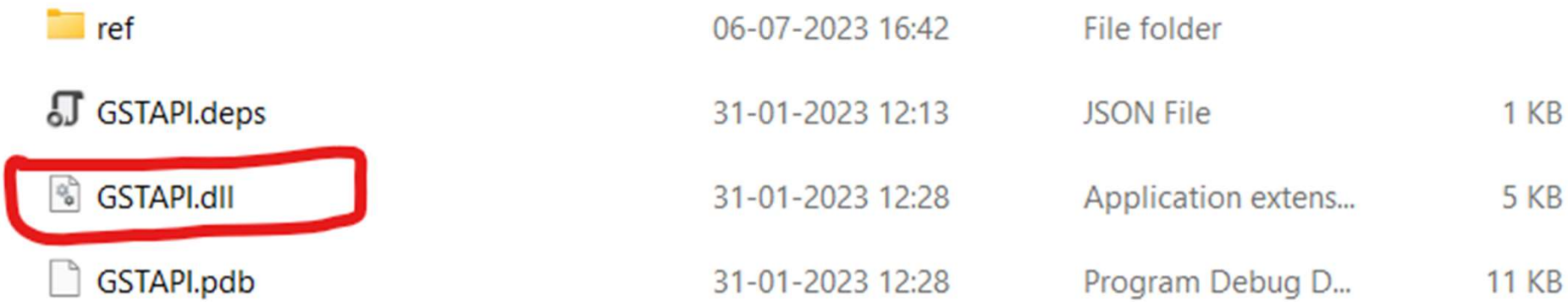
      public double CalcGstAmount()
      {
         var gstAmt = this.ProAmount / 100 *
this.Per;
         return gstAmt;
      }
```

```
      public double CalcTotalAmount()
      {
         var total = this.ProAmount + CalcGstAmount();
         return total;
      }

      public double CalcCompositGst()
      {
         var gstAmt = this.ProAmount / 100 * 1;
         var total = gstAmt + this.ProAmount;
         return total;
      }
   }
}
```

**Step 2-To confirm whether Shared Assembly (.dll) is created or not.**

- **Go to Project Folder ➔ Bin ➔Debug➔[net6.0]➔**



## Step 3-To Refer and use Shared Assembly.

- **Create Project of type Console App in Visual Studio and add code given below.**

```
using GSTAPI;

namespace RefGSTApiDemo
{
   class Program
   {
      static void Main()
      {
         GST gst = new GST(10000,10);
        var gstAmt= gst.CalcGstAmount();

         var total = gst.CalcTotalAmount();

         Console.WriteLine($"GST Amount:{gstAmt}\tTotal Amount:{total}");

       var CompositGst=  gst.CalcCompositGst();
         Console.WriteLine($"Composit GST:{CompositGst}");
      }
   }
}
```

# References


https://learn.microsoft.com/en-us/dotnet/standard/assembly/

https://www.codeguru.com/dotnet/dot-net-assemblies/

https://www.c-sharpcorner.com/uploadfile/5eae74/creating-and-using-shared-assembly/

https://www.c-sharpcorner.com/UploadFile/72d20e/concept-of-shared-assembly-in-net/

https://learn.microsoft.com/en-us/nuget/quickstart/create-and-publish-a-package-using-visual-studio-net-framework

https://learn.microsoft.com/en-us/dotnet/core/extensions/create-satellite-assemblies

https://www.c-sharpcorner.com/UploadFile/dacca2/register-your-assembly-in-gac-using-gacutil-exe/